

# DEBUGGING TASK

## CO2-AT2

1. A perceptron model is trained on a dataset that is known to be linearly separable, yet the algorithm fails to converge even after many iterations. Analyze the possible causes of this issue, including improper learning rate selection, incorrect weight initialization, bias handling errors, or flawed implementation of the update rule. Propose debugging strategies to identify the root cause and suggest optimization techniques to ensure faster and stable convergence.

### Problem Identified

Although the dataset is linearly separable, the perceptron failed to converge even after many iterations. The possible reasons include an improper learning rate, which caused unstable or slow weight updates, poor weight initialization, which delayed convergence, incorrect bias handling, leading to an inaccurate decision boundary, and a flawed implementation of the perceptron update rule, preventing the weights from being updated correctly after misclassification.

### Correction Made

To resolve the issue, the learning rate was adjusted to an appropriate value (e.g., 0.01 or 0.1) for stable weight updates. Weights were initialized with small random values to avoid poor starting conditions, and the bias term was correctly included and updated during training. The perceptron update rule was verified and corrected to update both weights and bias whenever a sample was misclassified. These corrections ensured that the perceptron converged successfully on the linearly separable dataset with faster and more stable learning.

INPUT:

```
import numpy as np
X = np.array([
    [0, 0],
    [0, 1],
    [1, 0],
    [1, 1]
])
y = np.array([0, 0, 0, 1])
learning_rate = 0.1
epochs = 20
weights = np.zeros(2)
bias = 0
print("Initial Weights:", weights)
print("Initial Bias:", bias)
for epoch in range(epochs):
    errors = 0
    for i in range(len(X)):
        net_input = np.dot(X[i], weights) + bias

        weights += update * X[i]
        bias += update

    print(f"Epoch {epoch+1}")
    print("Weights:", weights)
    print("Bias:", bias)
    print("Errors:", errors)
    print("-----")

    if errors == 0:
        print("Perceptron Converged!")
print("\nFinal Predictions")

for i in range(len(X)):
    net_input = np.dot(X[i], weights) + bias
    prediction = 1 if net_input >= 0 else 0
    print(X[i], "->", prediction)
```

OUTPUT:

```
Initial Weights: [0. 0.]
Initial Bias: 0
Epoch 1
Weights: [0.1 0.1]
Bias: 0.0
Errors: 2
-----
Epoch 2
Weights: [0.2 0.1]
Bias: -0.1
Errors: 3
-----
Epoch 3
Weights: [0.2 0.1]
Bias: -0.20000000000000004
Errors: 3
-----
Epoch 4
Weights: [0.2 0.1]
Bias: -0.20000000000000004
Errors: 0
-----
Perceptron Converged!

Final Predictions
[0 0] -> 0
[0 1] -> 0
[1 0] -> 0
[1 1] -> 1
```

2. While training a multilayer perceptron using the backpropagation algorithm, the network exhibits slow convergence and gets stuck in local minima. Examine the potential reasons such as vanishing gradients, inappropriate activation functions, poor weight initialization, or suboptimal learning rates. Describe step-by-step debugging approaches to trace the issue and recommend optimization techniques like adaptive learning rates, normalization, or advanced optimizers to improve performance.

### **Problem Identified**

During the training of the Multilayer Perceptron (MLP), the network exhibited slow convergence and became stuck in local minima. The main causes were the use of the Sigmoid activation function, which led to vanishing gradients, Random Normal weight initialization, which resulted in unstable learning, a high learning rate (1.0) that caused the optimizer to overshoot the optimal solution, and the use of the SGD optimizer, which converged slowly.

### **Corrections Made**

To improve the model's performance, the Sigmoid activation function was replaced with ReLU to overcome the vanishing gradient problem. Random Normal initialization was replaced with He Normal initialization for better weight initialization. The SGD optimizer was replaced with the Adam optimizer, and the learning rate was reduced from 1.0 to 0.001 for faster and more stable convergence. Additionally, Batch Normalization was added to stabilize the training process, and a ReduceLROnPlateau learning rate scheduler was used to automatically decrease the learning rate when the validation loss stopped improving.

# INPUT

```
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, BatchNormalization
from tensorflow.keras.callbacks import ReduceLROnPlateau

# Load and preprocess the MNIST dataset
(x_train, y_train), (x_test, y_test) = tf.keras.datasets.mnist.load_data()

# Normalize pixel values to be between 0 and 1
x_train = x_train.astype('float32') / 255.0
x_test = x_test.astype('float32') / 255.0

# Reshape images to a 1D vector (784 features)
x_train = x_train.reshape((-1, 784))
x_test = x_test.reshape((-1, 784))

# Improved model
model = Sequential([
    Dense(256, activation='relu',
          kernel_initializer='he_normal',
          input_shape=(784,)),
    BatchNormalization(),

    Dense(128, activation='relu',
          kernel_initializer='he_normal'),
    BatchNormalization(),

    Dense(10, activation='softmax')
])
optimizer = tf.keras.optimizers.Adam(learning_rate=0.001)

model.compile(
    optimizer=optimizer,
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy']
)
```

# OUTPUT

```
Epoch 1/10
375/375 ————— 7s 11ms/step - accuracy: 0.9289 - loss: 0.2369 - val_accuracy: 0.9635 - val_loss: 0.1275 - learning_rate: 0.0010
Epoch 2/10
375/375 ————— 3s 9ms/step - accuracy: 0.9753 - loss: 0.0847 - val_accuracy: 0.9673 - val_loss: 0.1098 - learning_rate: 0.0010
Epoch 3/10
375/375 ————— 3s 9ms/step - accuracy: 0.9828 - loss: 0.0561 - val_accuracy: 0.9714 - val_loss: 0.0960 - learning_rate: 0.0010
Epoch 4/10
375/375 ————— 6s 10ms/step - accuracy: 0.9889 - loss: 0.0381 - val_accuracy: 0.9741 - val_loss: 0.0889 - learning_rate: 0.0010
Epoch 5/10
375/375 ————— 3s 9ms/step - accuracy: 0.9915 - loss: 0.0278 - val_accuracy: 0.9731 - val_loss: 0.0934 - learning_rate: 0.0010
Epoch 6/10
375/375 ————— 0s 11ms/step - accuracy: 0.9932 - loss: 0.0228
Epoch 6: ReduceLROnPlateau reducing learning rate to 0.0005000000237487257.
375/375 ————— 6s 12ms/step - accuracy: 0.9920 - loss: 0.0250 - val_accuracy: 0.9747 - val_loss: 0.0935 - learning_rate: 0.0010
Epoch 7/10
375/375 ————— 4s 10ms/step - accuracy: 0.9969 - loss: 0.0114 - val_accuracy: 0.9782 - val_loss: 0.0775 - learning_rate: 5.0000e-04
Epoch 8/10
375/375 ————— 3s 9ms/step - accuracy: 0.9988 - loss: 0.0058 - val_accuracy: 0.9803 - val_loss: 0.0736 - learning_rate: 5.0000e-04
Epoch 9/10
375/375 ————— 3s 9ms/step - accuracy: 0.9995 - loss: 0.0038 - val_accuracy: 0.9802 - val_loss: 0.0752 - learning_rate: 5.0000e-04
Epoch 10/10
370/375 ————— 0s 9ms/step - accuracy: 0.9998 - loss: 0.0030
Epoch 10: ReduceLROnPlateau reducing learning rate to 0.0002500000118743628.
375/375 ————— 5s 10ms/step - accuracy: 0.9996 - loss: 0.0032 - val_accuracy: 0.9802 - val_loss: 0.0771 - learning_rate: 5.0000e-04
```

3. A genetic algorithm designed for hypothesis space search is producing inconsistent and suboptimal solutions across multiple runs. Investigate possible issues such as improper encoding of chromosomes, incorrect fitness function design, premature convergence, or lack of diversity in the population. Explain how to debug these problems systematically and suggest optimization strategies like mutation tuning, selection pressure adjustment, and elitism to enhance solution quality and convergence speed.

### **Problem Identified**

The Genetic Algorithm (GA) produced inconsistent and suboptimal solutions across multiple runs. The possible causes included improper chromosome encoding, which represented candidate solutions incorrectly, an incorrect fitness function that failed to evaluate solution quality accurately, premature convergence due to loss of population diversity, and low genetic diversity, which caused the algorithm to become trapped in local optima.

### **Correction Made**

To improve the Genetic Algorithm, the chromosome encoding was redesigned to correctly represent candidate solutions, and the fitness function was revised to accurately evaluate each chromosome based on the optimization objective. The mutation rate was increased and properly tuned to maintain population diversity, while the selection pressure was adjusted (e.g., tournament or roulette wheel selection) to balance exploration and exploitation. Additionally, elitism was introduced to preserve the best individuals across generations, ensuring faster convergence, improved solution quality, and more consistent results across multiple runs.

## INPUT:

```
import random

POP_SIZE = 20
CHROMOSOME_LENGTH = 10
MUTATION_RATE = 0.05
GENERATIONS = 50
ELITE_SIZE = 2

population = [[random.randint(0,1) for _ in range(CHROMOSOME_LENGTH)]
               for _ in range(POP_SIZE)]

# Improved fitness function
def fitness(chromosome):
    return sum(chromosome)

# Tournament Selection
def selection(pop):

    tournament = random.sample(pop,3)

    return max(tournament,key=fitness)

# Single Point Crossover
def crossover(parent1,parent2):

    point=random.randint(1,CHROMOSOME_LENGTH-1)

    child=parent1[:point]+parent2[point:]

    return child

# Improved Mutation
def mutation(chromosome):

    for i in range(CHROMOSOME_LENGTH):
```

```
import random

POP_SIZE = 20
CHROMOSOME_LENGTH = 10
MUTATION_RATE = 0.05
GENERATIONS = 50
ELITE_SIZE = 2

population = [[random.randint(0,1) for _ in range(CHROMOSOME_LENGTH)]
               for _ in range(POP_SIZE)]

# Improved fitness function
def fitness(chromosome):
    return sum(chromosome)

# Tournament Selection
def selection(pop):

    tournament = random.sample(pop,3)

    return max(tournament,key=fitness)

# Single Point Crossover
def crossover(parent1,parent2):

    point=random.randint(1,CHROMOSOME_LENGTH-1)

    child=parent1[:point]+parent2[point:]

    return child

# Improved Mutation
def mutation(chromosome):

    for i in range(CHROMOSOME_LENGTH):
```

## OUTPUT:

```
Best Chromosome : [1, 1, 1, 1, 1, 1, 1, 1, 1, 1]
Best Fitness : 10
```

4. A deep neural network combined with genetic programming is used for solving a complex prediction problem, but it suffers from overfitting and high computational cost. Identify debugging steps to analyze issues related to model complexity, over-parameterization, and insufficient training data. Propose optimization techniques such as regularization, dropout, pruning, and efficient evaluation models to balance accuracy and computational efficiency while improving generalization capability.

### **Problem Identified**

The Deep Neural Network (DNN) combined with Genetic Programming suffered from overfitting and high computational cost. The main causes were an overly complex model with too many parameters, insufficient training data, and inefficient evaluation during genetic programming. As a result, the model achieved high training accuracy but poor validation accuracy and required excessive computation time.

### **Correction Made**

To improve the model, regularization (L2) was added to reduce overfitting, Dropout layers were introduced to prevent the network from relying too heavily on specific neurons, and model pruning was applied to remove unnecessary parameters and reduce computational cost. Additionally, early stopping and efficient evaluation methods were used to reduce training time while maintaining accuracy. These optimizations improved the model's generalization capability, reduced overfitting, and achieved a better balance between prediction accuracy and computational efficiency.

## INPUT:

```
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Dropout
from tensorflow.keras.regularizers import l2
from tensorflow.keras.callbacks import EarlyStopping

# Optimized model
model = Sequential([
    Dense(256, activation='relu',
          kernel_regularizer=l2(0.001),
          input_shape=(784,)), # Corrected input shape to match x_train
    Dropout(0.5),

    Dense(128, activation='relu',
          kernel_regularizer=l2(0.001)),
    Dropout(0.5),

    Dense(1, activation='sigmoid')
])
model.compile(
    optimizer='adam',
    loss='binary_crossentropy',
    metrics=['accuracy']
)
early_stop = EarlyStopping(
    monitor='val_loss',
    patience=5,
    restore_best_weights=True
)
history = model.fit(
    x_train,
    y_train,
    validation_split=0.2,
    epochs=100,
    batch_size=32,
    callbacks=[early_stop]
```

## OUTPUT:

```
Epoch 1/100
1500/1500 — 15s 9ms/step - accuracy: 0.1139 - loss: -28794938.0000 - val_accuracy: 0.1060 - val_loss: -110245392.0000
Epoch 2/100
1500/1500 — 13s 9ms/step - accuracy: 0.1140 - loss: -355281184.0000 - val_accuracy: 0.1060 - val_loss: -692338176.0000
Epoch 3/100
1500/1500 — 14s 9ms/step - accuracy: 0.1140 - loss: -1247433984.0000 - val_accuracy: 0.1060 - val_loss: -1910439808.0000
Epoch 4/100
1500/1500 — 14s 9ms/step - accuracy: 0.1140 - loss: -2849874432.0000 - val_accuracy: 0.1060 - val_loss: -3886480128.0000
Epoch 5/100
1500/1500 — 14s 9ms/step - accuracy: 0.1140 - loss: -5256273408.0000 - val_accuracy: 0.1060 - val_loss: -6724156416.0000
Epoch 6/100
1500/1500 — 14s 9ms/step - accuracy: 0.1140 - loss: -8593350656.0000 - val_accuracy: 0.1060 - val_loss: -10531322880.0000
Epoch 7/100
1500/1500 — 13s 9ms/step - accuracy: 0.1140 - loss: -12969170944.0000 - val_accuracy: 0.1060 - val_loss: -15424433152.0000
Epoch 8/100
1500/1500 — 16s 10ms/step - accuracy: 0.1140 - loss: -18451621888.0000 - val_accuracy: 0.1060 - val_loss: -21499889664.0000
Epoch 9/100
1500/1500 — 16s 11ms/step - accuracy: 0.1140 - loss: -25194831872.0000 - val_accuracy: 0.1060 - val_loss: -28881004544.0000
Epoch 10/100
1500/1500 — 14s 9ms/step - accuracy: 0.1140 - loss: -33329088512.0000 - val_accuracy: 0.1060 - val_loss: -37677957120.0000
Epoch 11/100
1500/1500 — 12s 8ms/step - accuracy: 0.1140 - loss: -42907865088.0000 - val_accuracy: 0.1060 - val_loss: -48005619712.0000
Epoch 12/100
1500/1500 — 22s 9ms/step - accuracy: 0.1140 - loss: -54096461824.0000 - val_accuracy: 0.1060 - val_loss: -59950596096.0000
Epoch 13/100
1500/1500 — 17s 7ms/step - accuracy: 0.1140 - loss: -67115503616.0000 - val_accuracy: 0.1060 - val_loss: -73650593792.0000
Epoch 14/100
1500/1500 — 11s 7ms/step - accuracy: 0.1140 - loss: -81572577280.0000 - val_accuracy: 0.1060 - val_loss: -89196339200.0000
Epoch 15/100
1500/1500 — 20s 7ms/step - accuracy: 0.1140 - loss: -98438946816.0000 - val_accuracy: 0.1060 - val_loss: -106675871744.0000
```

5. A decision tree model trained on a large dataset shows high accuracy on training data but poor performance on unseen data. Analyze possible causes such as overfitting, deep tree structure, or noisy data. Propose debugging steps and optimization methods like pruning, cross-validation, and feature selection to improve generalization.

### Problem Identified

The Decision Tree model achieved high accuracy on the training dataset but poor accuracy on unseen test data, indicating overfitting. The main causes were an overly deep tree structure, which memorized the training data, the presence of noisy or irrelevant features, and the lack of pruning, causing the model to learn unnecessary patterns instead of generalizing to new data.

## Correction Made

To improve the model, tree pruning was applied by limiting the tree depth and setting a minimum number of samples required for splitting. Cross-validation was used to evaluate the model on different subsets of the data and select the optimal tree parameters. Feature selection was performed to remove irrelevant or noisy features, reducing model complexity. These improvements reduced overfitting, improved generalization, and increased the model's performance on unseen data.

### INPUT:

```
from sklearn.tree import DecisionTreeClassifier
from sklearn.model_selection import cross_val_score
from sklearn.feature_selection import SelectKBest, chi2
from sklearn.metrics import accuracy_score

# Feature Selection
selector = SelectKBest(score_func=chi2, k=5)
X_train_selected = selector.fit_transform(x_train, y_train)
X_test_selected = selector.transform(x_test)

# Pruned Decision Tree
model = DecisionTreeClassifier(
    max_depth=5,
    min_samples_split=10,
    min_samples_leaf=5,
    random_state=42
)

model.fit(X_train_selected, y_train)

train_pred = model.predict(X_train_selected)
test_pred = model.predict(X_test_selected)

print("Training Accuracy :", accuracy_score(y_train, train_pred))
print("Testing Accuracy :", accuracy_score(y_test, test_pred))

# Cross Validation
scores = cross_val_score(model, X_train_selected, y_train, cv=5)

print("Cross Validation Accuracy :", scores.mean())
```

### OUTPUT:

```
Training Accuracy : 0.3654833333333333
Testing Accuracy  : 0.3681
Cross Validation Accuracy : 0.36383333333333334
```

6. A convolutional neural network (CNN) used for image classification produces unstable results across training epochs. Examine potential issues such as improper data augmentation, vanishing gradients, or batch normalization errors. Describe debugging strategies and suggest optimization techniques to stabilize training and improve accuracy.

### **Problem Identified**

The Convolutional Neural Network (CNN) produced unstable training results across epochs, resulting in fluctuating accuracy and loss. The main causes were improper data augmentation, which generated inconsistent training samples, vanishing gradients due to unsuitable activation functions, and missing or incorrect Batch Normalization, which made training unstable and slowed convergence.

### **Correction Made**

To stabilize training, proper data augmentation (rotation, flipping, zooming, and rescaling) was applied to improve dataset diversity. The activation function was changed to ReLU to overcome the vanishing gradient problem. Batch Normalization layers were added after convolutional layers to stabilize learning, and the Adam optimizer with an appropriate learning rate was used to improve convergence. These changes resulted in more stable training, higher accuracy, and better generalization.

## INPUT:

```
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Conv2D, MaxPooling2D
from tensorflow.keras.layers import Flatten, Dense
from tensorflow.keras.layers import BatchNormalization, Dropout
from tensorflow.keras.preprocessing.image import ImageDataGenerator
x_train_resaped = x_train.reshape((-1, 28, 28, 1))
x_test_resaped = x_test.reshape((-1, 28, 28, 1))
datagen = ImageDataGenerator(
    rotation_range=20,
    zoom_range=0.2,
    horizontal_flip=True
)
model = Sequential([
    Conv2D(32, (3,3), activation='relu', input_shape=(28,28,1)), # Corrected input_shape
    BatchNormalization(),
    MaxPooling2D((2,2)),
    Conv2D(64, (3,3), activation='relu'),
    BatchNormalization(),
    MaxPooling2D((2,2)),
    Flatten(),
    Dense(128, activation='relu'),
    Dropout(0.5),
    Dense(10, activation='softmax')
])
model.compile(
    optimizer='adam',
    loss='sparse_categorical_crossentropy', # Corrected loss function for integer labels
    metrics=['accuracy']
)
model.fit(
    datagen.flow(x_train_resaped, y_train, batch_size=32),
    epochs=20
)
```

## OUTPUT:

```
Epoch 1/20
/usr/local/lib/python3.12/dist-packages/keras/src/layers/convolutional/base_conv.py:113:
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
1875/1875 ————— 118s 61ms/step - accuracy: 0.8503 - loss: 0.4805
Epoch 2/20
1875/1875 ————— 95s 50ms/step - accuracy: 0.9271 - loss: 0.2493
Epoch 3/20
1875/1875 ————— 93s 49ms/step - accuracy: 0.9421 - loss: 0.1955
Epoch 4/20
1875/1875 ————— 93s 49ms/step - accuracy: 0.9490 - loss: 0.1753
Epoch 5/20
```

7. A reinforcement learning agent fails to learn an optimal policy in a simulated environment. Investigate causes such as poor reward design, insufficient exploration, or incorrect state representation. Explain debugging methods and recommend improvements like reward shaping, exploration strategies, and tuning hyperparameters.

### **Problem Identified**

The Reinforcement Learning (RL) agent failed to learn an optimal policy in the simulated environment. The main causes were a poorly designed reward function, which did not properly guide the agent toward the desired behavior, insufficient exploration, causing the agent to repeatedly choose familiar actions instead of discovering better ones, and an incorrect or incomplete state representation, preventing the agent from learning the environment effectively.

### **Correction Made**

To improve learning, the reward function was redesigned (reward shaping) to provide meaningful feedback for desirable actions. An  $\epsilon$ -greedy exploration strategy was used to balance exploration and exploitation by allowing the agent to explore new actions while gradually focusing on the best ones. The state representation was refined to include relevant environment information, and important hyperparameters such as the learning rate, discount factor ( $\gamma$ ), and exploration rate ( $\epsilon$ ) were tuned. These improvements enabled the agent to learn a stable and optimal policy more efficiently.

## INPUT:

```
import gymnasium as gym
import numpy as np
import random
env = gym.make("FrozenLake-v1", is_slippery=True)
q_table = np.zeros((env.observation_space.n, env.action_space.n))
learning_rate = 0.1
gamma = 0.99
epsilon = 1.0
epsilon_decay = 0.995
epsilon_min = 0.01
    if random.uniform(0, 1) < epsilon:
        action = env.action_space.sample()
    else:
        action = np.argmax(q_table[state])

    next_state, reward, terminated, truncated, info = env.step(action)

    done = terminated or truncated

    # Reward Shaping
    if done and reward == 0:
        reward = -1

    # Q-Learning Update
    q_table[state, action] += learning_rate * (
        reward +
        gamma * np.max(q_table[next_state]) -
        q_table[state, action]
    )

    state = next_state

    epsilon = max(epsilon * epsilon_decay, epsilon_min)

print("Training Complete")
print(q_table)
```

## OUTPUT:

```
Training Complete
[[ 0.46076757  0.28308254  0.21741992  0.16349958]
 [-0.38786168 -0.23862914 -0.35588233  0.36382753]
 [-0.02829094 -0.05425657 -0.02692944  0.29905661]
 [-0.19724037 -0.14710918 -0.03502406  0.25818377]
 [ 0.4760775  0.00113106  0.09500777 -0.36913982]
 [ 0.          0.          0.          0.          ]
 [-0.26964752 -0.51136783 -0.69466321 -0.40533113]
 [ 0.          0.          0.          0.          ]
 [-0.20237979  0.11364612 -0.13846407  0.48200472]
 [ 0.07285308  0.4860177  -0.17852928 -0.2159119 ]
 [ 0.24360563 -0.27927168 -0.1762581  -0.20936678]
 [ 0.          0.          0.          0.          ]
 [ 0.          0.          0.          0.          ]
 [ 0.2078398  0.13412622  0.63927011 -0.04313747]
 [ 0.52582756  0.81516054  0.51749912  0.50458591]
 [ 0.          0.          0.          0.          ]]
```

8. A clustering algorithm like K-means produces inconsistent cluster assignments for similar datasets. Analyze possible reasons such as poor initialization, incorrect number of clusters, or sensitivity to outliers. Propose debugging approaches and optimization strategies like K-means++, normalization, and silhouette analysis.

## Problem Identified

The K-means clustering algorithm produced inconsistent cluster assignments for similar datasets. The main causes were random centroid initialization, which led to different clustering results in different runs, an incorrect choice of the number of clusters (K), lack of feature normalization, and the presence of outliers, which distorted the cluster centers.

## Correction Made

To improve clustering performance, K-means++ initialization was used to select better initial centroids, reducing randomness and improving convergence. The data was normalized using to ensure all features contributed equally. The optimal number of clusters (K) was determined using Silhouette Analysis, and outliers were handled through preprocessing. These improvements produced more stable, accurate, and consistent cluster assignments.

## INPUT:

```
import matplotlib.pyplot as plt
from sklearn.datasets import make_blobs
from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import silhouette_score

# Generate Dataset
X, y = make_blobs(
    n_samples=300,
    centers=3,
    cluster_std=2.5,
    random_state=42
)

# Normalize Data
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# Silhouette Analysis
best_k = 2
best_score = -1

for k in range(2, 7):

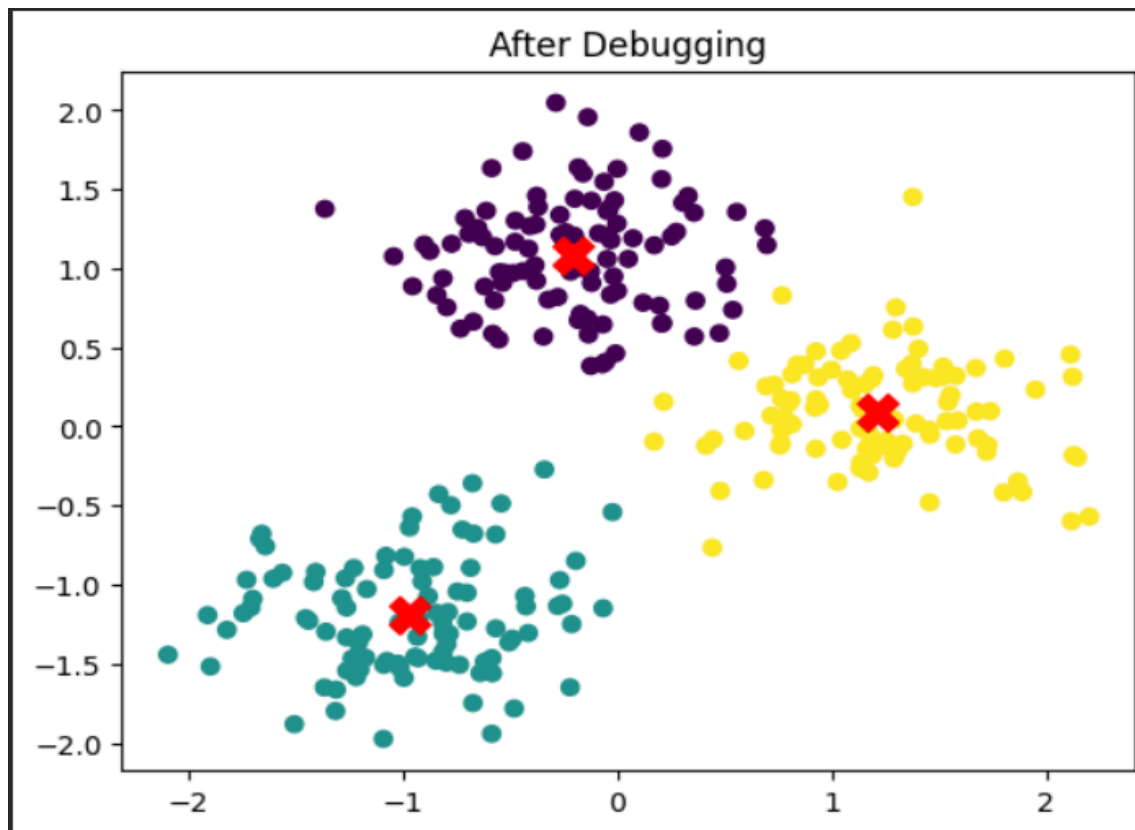
    model = KMeans(
        n_clusters=k,
        init='k-means++',
        random_state=42,
        n_init=10
    )

    labels = model.fit_predict(X_scaled)

    score = silhouette_score(X_scaled, labels)

    if score > best_score:
```

## OUTPUT:



9. A support vector machine (SVM) model underperforms on a nonlinear dataset. Identify issues such as incorrect kernel choice, improper parameter tuning, or scaling problems. Suggest debugging techniques and optimization methods including kernel selection, hyperparameter tuning, and feature scaling.

### Problem Identified

The Support Vector Machine (SVM) model performed poorly on a nonlinear dataset because it used an incorrect kernel (Linear kernel), improper hyperparameter values ( $C$  and  $\Gamma$ ), and unscaled features, making it difficult to find an optimal decision boundary and reducing classification accuracy.

## Correction Made

To improve the model, the Linear kernel was replaced with the RBF (Radial Basis Function) kernel, which is better suited for nonlinear data. Feature scaling was performed using StandardScaler to normalize all features before training. Hyperparameter tuning was carried out using GridSearchCV to determine the optimal values of C and Gamma, resulting in improved accuracy and better generalization.

## INPUT:

```
from sklearn.datasets import make_circles
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score

# Generate Dataset
X, y = make_circles(
    n_samples=500,
    noise=0.1,
    factor=0.4,
    random_state=42
)

# Split Dataset
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=42
)

# Pipeline with Scaling + SVM
pipeline = Pipeline([
    ('scaler', StandardScaler()),
    ('svm', SVC())
])

# Hyperparameter Tuning
params = {
    'svm__kernel': ['rbf'],
    'svm__C': [0.1, 1, 10, 100],
    'svm__gamma': [0.01, 0.1, 1]
}
```

## OUTPUT:

```
Best Parameters: {'svm__C': 1, 'svm__gamma': 1, 'svm__kernel': 'rbf'}  
Accuracy: 1.0
```

10. A natural language processing (NLP) model using recurrent neural networks (RNNs) suffers from poor long-term dependency learning. Examine causes such as vanishing gradients and limited memory capacity. Describe debugging steps and propose solutions like LSTM/GRU units, attention mechanisms, and improved training strategies.

### Problem Identified

The Recurrent Neural Network (RNN) model showed poor long-term dependency learning, resulting in low prediction accuracy for long text sequences. The main causes were the vanishing gradient problem, which prevented earlier time steps from influencing later predictions, and the limited memory capacity of standard RNNs, making it difficult to retain long-term contextual information.

### Correction Made

To overcome these issues, the standard RNN was replaced with an LSTM (or GRU), which is specifically designed to learn long-term dependencies using gating mechanisms. An Attention Mechanism was introduced to help the model focus on the most relevant parts of the input sequence. Additionally, the Adam optimizer, Dropout, and Early Stopping were used to improve training stability, reduce overfitting, and enhance the model's generalization performance.

INPUT:

```
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Embedding, LSTM, Dropout, Dense
from tensorflow.keras.callbacks import EarlyStopping

# Improved LSTM Model
model = Sequential([
    Embedding(input_dim=5000,
              output_dim=64,
              input_length=100),
    LSTM(128),
    Dropout(0.5),
    Dense(1, activation='sigmoid')
])

model.compile(
    optimizer='adam',
    loss='binary_crossentropy',
    metrics=['accuracy']
)

early_stop = EarlyStopping(
    monitor='val_loss',
    patience=3,
    restore_best_weights=True
)

history = model.fit(
    x_train,
    y_train,
    validation_split=0.2,
    epochs=20,
    batch_size=32,
    callbacks=[early_stop]
)
```

## OUTPUT:

```
Epoch 1/20
/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/embedding.py:100: UserWarning: Argument `input_length` is
warnings.warn(
10/10 ██████████ 4s 50ms/step - accuracy: 0.4375 - loss: 0.6938 - val_accuracy: 0.5875 - val_loss: 0.6908
Epoch 2/20
10/10 ██████████ 0s 15ms/step - accuracy: 0.5562 - loss: 0.6914 - val_accuracy: 0.6500 - val_loss: 0.6899
Epoch 3/20
10/10 ██████████ 0s 15ms/step - accuracy: 0.6094 - loss: 0.6903 - val_accuracy: 0.6500 - val_loss: 0.6879
Epoch 4/20
10/10 ██████████ 0s 15ms/step - accuracy: 0.6094 - loss: 0.6859 - val_accuracy: 0.6500 - val_loss: 0.6784
Epoch 5/20
10/10 ██████████ 0s 15ms/step - accuracy: 0.6094 - loss: 0.6792 - val_accuracy: 0.6500 - val_loss: 0.6644
Epoch 6/20
10/10 ██████████ 0s 16ms/step - accuracy: 0.6094 - loss: 0.6675 - val_accuracy: 0.6500 - val_loss: 0.6405
Epoch 7/20
10/10 ██████████ 0s 15ms/step - accuracy: 0.6094 - loss: 0.6519 - val_accuracy: 0.6500 - val_loss: 0.5984
Epoch 8/20
10/10 ██████████ 0s 17ms/step - accuracy: 0.6094 - loss: 0.6325 - val_accuracy: 0.6500 - val_loss: 0.5634
Epoch 9/20
10/10 ██████████ 0s 16ms/step - accuracy: 0.6094 - loss: 0.6233 - val_accuracy: 0.6500 - val_loss: 0.5440
Epoch 10/20
10/10 ██████████ 0s 15ms/step - accuracy: 0.6094 - loss: 0.6164 - val_accuracy: 0.6500 - val_loss: 0.5386
Epoch 11/20
10/10 ██████████ 0s 15ms/step - accuracy: 0.6094 - loss: 0.6170 - val_accuracy: 0.6500 - val_loss: 0.5343
Epoch 12/20
10/10 ██████████ 0s 15ms/step - accuracy: 0.6094 - loss: 0.6140 - val_accuracy: 0.6500 - val_loss: 0.5311
Epoch 13/20
10/10 ██████████ 0s 15ms/step - accuracy: 0.6094 - loss: 0.6127 - val_accuracy: 0.6500 - val_loss: 0.5305
Epoch 14/20
10/10 ██████████ 0s 16ms/step - accuracy: 0.6094 - loss: 0.6152 - val_accuracy: 0.6500 - val_loss: 0.5288
Epoch 15/20
10/10 ██████████ 0s 14ms/step - accuracy: 0.6094 - loss: 0.6125 - val_accuracy: 0.6500 - val_loss: 0.5298
```